

Tagging and Extraction Using OpenAI functions

```
In [1]: import os
import openai

from dotenv import load_dotenv, find_dotenv
_ = load_dotenv(find_dotenv()) # read local .env file
openai.api_key = os.environ['OPENAI_API_KEY']
```

```
In [2]: from typing import List
from pydantic import BaseModel, Field
from langchain.utils.openai_functions import convert_pydantic_to_openai_func
```

```
In [3]: class Tagging(BaseModel):
        """Tag the piece of text with particular info."""
        sentiment: str = Field(description="sentiment of text, should be `pos`,
        language: str = Field(description="language of text (should be ISO 639-1
```

```
In [4]: convert_pydantic_to_openai_function(Tagging)
```

```
Out[4]: {'name': 'Tagging',
'description': 'Tag the piece of text with particular info.',
'parameters': {'title': 'Tagging',
'description': 'Tag the piece of text with particular info.',
'type': 'object',
'properties': {'sentiment': {'title': 'Sentiment',
'description': 'sentiment of text, should be `pos`, `neg`, or `neutral`',
'type': 'string'},
'language': {'title': 'Language',
'description': 'language of text (should be ISO 639-1 code)',
'type': 'string'}}},
'required': ['sentiment', 'language']}]}
```

```
In [5]: from langchain.prompts import ChatPromptTemplate
from langchain.chat_models import ChatOpenAI
```

```
In [6]: model = ChatOpenAI(temperature=0)
```

```
In [7]: tagging_functions = [convert_pydantic_to_openai_function(Tagging)]
```

```
In [8]: prompt = ChatPromptTemplate.from_messages([
        ("system", "Think carefully, and then tag the text as instructed"),
        ("user", "{input}")
    ])
```

```
In [9]: model_with_functions = model.bind(
        functions=tagging_functions,
```

```
function_call={"name": "Tagging"}
)
```

```
In [10]: tagging_chain = prompt | model_with_functions
```

```
In [11]: tagging_chain.invoke({"input": "I love langchain"})
```

```
Out[11]: AIMessage(content='', additional_kwargs={'function_call': {'name': 'Tagging', 'arguments': {'sentiment': 'pos', 'language': 'en'}}})
```

```
In [12]: tagging_chain.invoke({"input": "non mi piace questo cibo"})
```

```
Out[12]: AIMessage(content='', additional_kwargs={'function_call': {'name': 'Tagging', 'arguments': {'sentiment': 'neg', 'language': 'it'}}})
```

```
In [13]: from langchain.output_parsers.openai_functions import JsonOutputFunctionsParser
```

```
In [14]: tagging_chain = prompt | model_with_functions | JsonOutputFunctionsParser()
```

```
In [15]: tagging_chain.invoke({"input": "non mi piace questo cibo"})
```

```
Out[15]: {'sentiment': 'neg', 'language': 'it'}
```

Extraction

Extraction is similar to tagging, but used for extracting multiple pieces of information.

```
In [16]: from typing import Optional
class Person(BaseModel):
    """Information about a person."""
    name: str = Field(description="person's name")
    age: Optional[int] = Field(description="person's age")
```

```
In [17]: class Information(BaseModel):
    """Information to extract."""
    people: List[Person] = Field(description="List of info about people")
```

```
In [18]: convert_pydantic_to_openai_function(Information)
```

```
Out[18]: {'name': 'Information',
          'description': 'Information to extract.',
          'parameters': {'title': 'Information',
                          'description': 'Information to extract.',
                          'type': 'object',
                          'properties': {'people': {'title': 'People',
                                                    'description': 'List of info about people',
                                                    'type': 'array',
                                                    'items': {'title': 'Person',
                                                            'description': 'Information about a person.',
                                                            'type': 'object',
                                                            'properties': {'name': {'title': 'Name',
                                                                    'description': "person's name",
                                                                    'type': 'string'},
                                                            'age': {'title': 'Age',
                                                                    'description': "person's age",
                                                                    'type': 'integer'}}},
                          'required': ['name']}}},
          'required': ['people']}}
```

```
In [19]: extraction_functions = [convert_pydantic_to_openai_function(Information)]
         extraction_model = model.bind(functions=extraction_functions, function_call=
```

```
In [20]: extraction_model.invoke("Joe is 30, his mom is Martha")
```

```
Out[20]: AIMessage(content='', additional_kwargs={'function_call': {'name': 'Information', 'arguments': '{"people":[{"name":"Joe","age":30}, {"name":"Martha"}]'}})
```

```
In [21]: prompt = ChatPromptTemplate.from_messages([
          ("system", "Extract the relevant information, if not explicitly provided"),
          ("human", "{input}")
        ])
```

```
In [22]: extraction_chain = prompt | extraction_model
```

```
In [23]: extraction_chain.invoke({"input": "Joe is 30, his mom is Martha"})
```

```
Out[23]: AIMessage(content='', additional_kwargs={'function_call': {'name': 'Information', 'arguments': '{"people":[{"name":"Joe","age":30}, {"name":"Martha"}]'}})
```

```
In [24]: extraction_chain = prompt | extraction_model | JsonOutputFunctionsParser()
```

```
In [25]: extraction_chain.invoke({"input": "Joe is 30, his mom is Martha"})
```

```
Out[25]: {'people': [{'name': 'Joe', 'age': 30}, {'name': 'Martha'}]}
```

```
In [26]: from langchain.output_parsers.openai_functions import JsonKeyOutputFunctions
```

```
In [27]: extraction_chain = prompt | extraction_model | JsonKeyOutputFunctionsParser()
```

```
In [28]: extraction_chain.invoke({"input": "Joe is 30, his mom is Martha"})
```

```
Out [28]: [{'name': 'Joe', 'age': 30}, {'name': 'Martha'}]
```

Doing it for real

We can apply tagging to a larger body of text.

For example, let's load this blog post and extract tag information from a sub-set of the text.

```
In [29]: from langchain.document_loaders import WebBaseLoader  
loader = WebBaseLoader("https://lilianweng.github.io/posts/2023-06-23-agent/  
documents = loader.load()
```

```
In [30]: doc = documents[0]
```

```
In [31]: page_content = doc.page_content[:10000]
```

```
In [32]: print(page_content[:1000])
```


Posts

Archive

Search

Tags

FAQ

emojisearch.app

LLM Powered Autonomous Agents

Date: June 23, 2023 | Estimated Reading Time: 31 min | Author: Lilian We
ng

Table of Contents

Agent System Overview

Component One: Planning

Task Decomposition

Self-Reflection

Component Two: Memory

Types of Memory

Maximum Inner Product Search (MIPS)

Component Three: Tool Use

Case Studies

Scientific Discovery Agent

Generative Agents Simulation

Proof-of-Concept Examples

Challenges

Citation

References

Building agents with LLM (large language model) as its core controller is a cool concept. Several proof-of-concepts demos, such as AutoGPT, GPT-Engineer and BabyAGI, serve as inspiring examples. The potentiality of LLM extends beyond generating well-written copies, stories, essays and programs; it can be framed as a powerful general

```
In [33]: class Overview(BaseModel):  
        """Overview of a section of text."""  
        summary: str = Field(description="Provide a concise summary of the content in the  
        language: str = Field(description="Provide the language that the content is written in  
        keywords: str = Field(description="Provide keywords related to the content")
```

```
In [34]: overview_tagging_function = [  
        convert_pydantic_to_openai_function(Overview)  
    ]
```

```

tagging_model = model.bind(
    functions=overview_tagging_function,
    function_call={"name": "Overview"}
)
tagging_chain = prompt | tagging_model | JsonOutputFunctionsParser()

```

```
In [35]: tagging_chain.invoke({"input": page_content})
```

```

Out[35]: {'summary': 'This article discusses building autonomous agents powered by LLM (large language model) as the core controller. It covers components like planning, memory, and tool use, along with challenges and references to related studies.',
          'language': 'English',
          'keywords': 'LLM, autonomous agents, planning, memory, tool use, challenges, references'}

```

```

In [36]: class Paper(BaseModel):
          """Information about papers mentioned."""
          title: str
          author: Optional[str]

          class Info(BaseModel):
              """Information to extract"""
              papers: List[Paper]

```

```

In [37]: paper_extraction_function = [
          convert_pydantic_to_openai_function(Info)
          ]
          extraction_model = model.bind(
              functions=paper_extraction_function,
              function_call={"name": "Info"}
          )
          extraction_chain = prompt | extraction_model | JsonKeyOutputFunctionsParser()

```

```
In [38]: extraction_chain.invoke({"input": page_content})
```

```
Out[38]: [{'title': 'LLM Powered Autonomous Agents', 'author': 'Lilian Weng'}]
```

```

In [39]: template = """A article will be passed to you. Extract from it all papers that are mentioned in the article.
          Do not extract the name of the article itself. If no papers are mentioned then return an empty list.
          Do not make up or guess ANY extra information. Only extract what exactly is mentioned in the article.

          prompt = ChatPromptTemplate.from_messages([
              ("system", template),
              ("human", "{input}")
          ])

```

```
In [40]: extraction_chain = prompt | extraction_model | JsonKeyOutputFunctionsParser()
```

```
In [41]: extraction_chain.invoke({"input": page_content})
```



```
Out[41]: [{'title': 'Chain of thought (CoT; Wei et al. 2022)'},
          {'title': 'Tree of Thoughts (Yao et al. 2023)'},
          {'title': 'LLM+P (Liu et al. 2023)'},
          {'title': 'ReAct (Yao et al. 2023)'},
          {'title': 'Reflexion (Shinn & Labash 2023)'},
          {'title': 'Chain of Hindsight (CoH; Liu et al. 2023)'},
          {'title': 'Algorithm Distillation (AD; Laskin et al. 2023)'}]
```

```
In [ ]: extraction_chain.invoke({"input": "hi"})
```

```
In [43]: from langchain.text_splitter import RecursiveCharacterTextSplitter
text_splitter = RecursiveCharacterTextSplitter(chunk_overlap=0)
```

```
In [44]: splits = text_splitter.split_text(doc.page_content)
```

```
In [45]: len(splits)
```

```
Out[45]: 14
```

```
In [46]: def flatten(matrix):
          flat_list = []
          for row in matrix:
              flat_list += row
          return flat_list
```

```
In [47]: flatten([[1, 2], [3, 4]])
```

```
Out[47]: [1, 2, 3, 4]
```

```
In [48]: print(splits[0])
```


Posts

Archive

Search

Tags

FAQ

emojisearch.app

LLM Powered Autonomous Agents

Date: June 23, 2023 | Estimated Reading Time: 31 min | Author: Lilian We
ng

Table of Contents

Agent System Overview

Component One: Planning

Task Decomposition

Self-Reflection

Component Two: Memory

Types of Memory

Maximum Inner Product Search (MIPS)

Component Three: Tool Use

Case Studies

Scientific Discovery Agent

Generative Agents Simulation

Proof-of-Concept Examples

Challenges

Citation

References

Building agents with LLM (large language model) as its core controller is a cool concept. Several proof-of-concepts demos, such as AutoGPT, GPT-Engineer and BabyAGI, serve as inspiring examples. The potentiality of LLM extends beyond generating well-written copies, stories, essays and programs; it can be framed as a powerful general problem solver.

Agent System Overview#

In a LLM-powered autonomous agent system, LLM functions as the agent's brain, complemented by several key components:

Planning

Subgoal and decomposition: The agent breaks down large tasks into smaller, manageable subgoals, enabling efficient handling of complex tasks.

Reflection and refinement: The agent can do self-criticism and self-reflection on past actions, learn from mistakes and refine them for future steps, thereby improving the quality of final results.

Memory

Short-term memory: I would consider all the in-context learning (See Prompt Engineering) as utilizing short-term memory of the model to learn.

Long-term memory: This provides the agent with the capability to retain and recall (infinite) information over extended periods, often by leveraging an external vector store and fast retrieval.

Tool use

The agent learns to call external APIs for extra information that is missing from the model weights (often hard to change after pre-training), including current information, code execution capability, access to proprietary information sources and more.

```
In [49]: from langchain.schema.runnable import RunnableLambda
```

```
In [50]: prep = RunnableLambda(  
    lambda x: [{"input": doc} for doc in text_splitter.split_text(x)]  
)
```

```
In [51]: prep.invoke("hi")
```

```
Out[51]: [{'input': 'hi'}]
```

```
In [52]: chain = prep | extraction_chain.map() | flatten
```

```
In [53]: chain.invoke(doc.page_content)
```

```

Out[53]: [{'title': 'AutoGPT'},
          {'title': 'GPT-Engineer'},
          {'title': 'BabyAGI'},
          {'title': 'Chain of thought'},
          {'title': 'Tree of Thoughts'},
          {'title': 'LLM+P'},
          {'title': 'ReAct'},
          {'title': 'Reflexion'},
          {'title': 'Chain of Hindsight (CoH; Liu et al. 2023)'},
          {'title': 'Algorithm Distillation (AD; Laskin et al. 2023)'},
          {'title': 'Laskin et al. 2023'},
          {'title': 'Miller 1956'},
          {'title': 'Duan et al. 2017'},
          {'title': 'Google Blog'},
          {'title': 'ann-benchmarks.com'},
          {'title': 'MRKL (Karpas et al. 2022)'},
          {'title': 'TALM (Tool Augmented Language Models; Parisi et al. 2022)'},
          {'title': 'Toolformer (Schick et al. 2023)'},
          {'title': 'HuggingGPT (Shen et al. 2023)'},
          {'title': 'API-Bank', 'author': 'Li et al. 2023'},
          {'title': 'ChemCrow', 'author': 'Bran et al. 2023'},
          {'title': 'Boiko et al. (2023)'},
          {'title': 'Park et al. 2023'},
          {'title': 'Sample Paper 1', 'author': 'Author A'},
          {'title': 'Sample Paper 2', 'author': 'Author B'},
          {'title': 'Paper A', 'author': 'Author A'},
          {'title': 'Paper B', 'author': 'Author B'},
          {'title': 'pytest'},
          {'title': 'dataclasses'},
          {'title': 'Chain of thought prompting elicits reasoning in large language models.'},
          {'title': 'Tree of Thoughts: Deliberate Problem Solving with Large Language Models.'},
          {'title': 'Chain of Hindsight Aligns Language Models with Feedback'},
          {'title': 'LLM+P: Empowering Large Language Models with Optimal Planning Proficiency'},
          {'title': 'ReAct: Synergizing reasoning and acting in language models.'},
          {'title': 'Reflexion: an autonomous agent with dynamic memory and self-reflection'},
          {'title': 'In-context Reinforcement Learning with Algorithm Distillation'},
          {'title': 'MRKL Systems A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning.'},
          {'title': 'API-Bank: A Benchmark for Tool-Augmented LLMs'},
          {'title': 'HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in HuggingFace'},
          {'title': 'ChemCrow: Augmenting large-language models with chemistry tools.'},
          {'title': 'Emergent autonomous scientific research capabilities of large language models.'},
          {'title': 'Generative Agents: Interactive Simulacra of Human Behavior.'}]

```

In []:

In []: